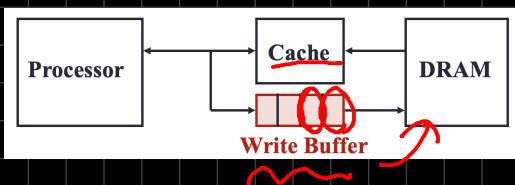


cache operation

	read	write
hit	①	③
miss	②	④



write-through

A scheme in which writes always update both the cache and the next lower level of the memory hierarchy, ensuring that data is always consistent between the two.

write-back A scheme that handles writes by updating values only to the block in the cache, then writing the modified block to the lower level of the hierarchy when the block is replaced.

write buffer A queue that holds data while the data is waiting to be written to memory.

①: read hit
→ 正常執行

②: read miss
→ CPU stall, 從 mem 抓 data 上 cache

③: write hit

→ write cache, 但 cache 存的是 mem 的 copy, 有 consistency 問題

Solution

(i) write through: 同步 update cache & mem

(ii) write back: 直到最後一刻 (須被 evict 出 cache 時)
⇒ 用 dirty-bit 來判斷被寫

* 加速: write buffer (batch)

④: write miss

→ CPU 要 write 的 data 不在 cache, 要去 mem 抓

Solution

搭配: write back *

(i) write allocation: 把 block 抓到 cache, 再寫

(ii) write around: 不抓進 cache, 直接寫 mem

搭配: write through *

★ 考試最快記法

把它記成兩個陣營:

「Cache 派」
Write allocate + Write-back
↓
能留 Cache 就留 Cache

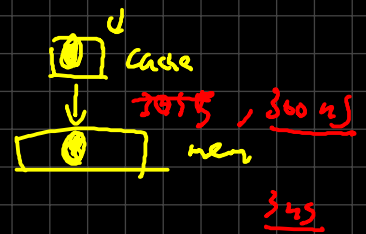
「Memory 派」
No-write-allocate + Write-through
↓
反正都要寫 Memory

所以兩者的核心思想一致:

Write allocate: 把資料留在 Cache。
Write-back: 修改也盡量留在 Cache。

Write-through + Write around

「反正 Memory 都要更新, miss 就不要浪費時間把 block 抓進來。」



- **Compulsory misses:** These are cache misses caused by the first access to a block that has never been in the cache. These are also called **cold-start misses**.
- **Capacity misses:** These are cache misses caused when the cache cannot contain all the blocks needed during execution of a program. Capacity misses occur when blocks are replaced and then later retrieved.
- **Conflict misses:** These are cache misses that occur in set-associative or direct-mapped caches when multiple blocks compete for the same set. Conflict misses are those misses in a direct-mapped or set-associative cache that are eliminated in a fully associative cache of the same size. These cache misses are also called **collision misses**.

Design change	Effect on miss rate	Possible negative performance effect
Increases <u>cache size</u>	Decreases <u>capacity misses</u>	May increase <u>access time</u>
Increases <u>associativity</u>	Decreases miss rate due to <u>conflict misses</u>	May increase <u>access time</u>
Increases <u>block size</u>	Decreases <u>miss rate</u> for a wide range of block sizes due to <u>spatial locality</u>	Increases <u>miss penalty</u> . Very large block could increase miss rate

FIGURE 5.38 Memory hierarchy design challenges.

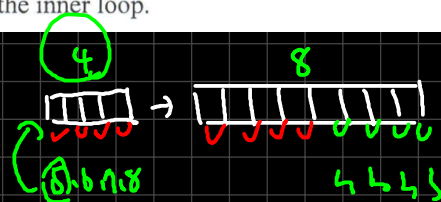
交大 100

#block ↓, competition ↑
(conflict miss)

3. Consider the cache design.

- Increasing block size can lower compulsory miss as well as miss penalty.
- The reason why increasing cache size can lower capacity miss is the available block number in the cache increases.
- Increasing block size within reasonable range can raise hit rate.
- If memory stores an integer array in a row-based manner, it is more efficient to traverse this array by setting row variable in the outer loop and column variable in the inner loop.

(a) block size ↑, compulsory miss ↓
miss penalty ↑



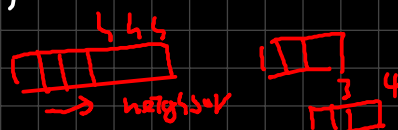
(b) cache size ↑, capacity miss ↓ (block size fixed)
(∵ available blocks ↑)



(c) block size ↑ (in reasonable size), hit rate ↑

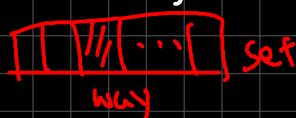
交大 104

Spatial locality

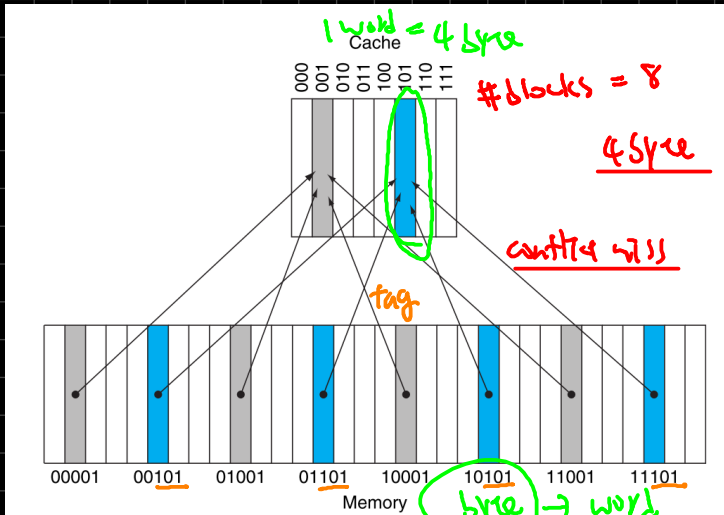


(e) Increase associativity may increase miss rate due to conflict misses

(e) associativity ↑, conflict miss ↓



Direct-mapped cache



∴ 有 $8 = 2^3$ 個 blocks
∴ 取 address 後 3-bit 做 Index

ex. block 5 (10101)

addr. 00101 mod 8 = 101
01101 mod 8 = 101
10101 mod 8 = 101
11101 mod 8 = 101

byte

1 word = 4 bytes

$4 = 2^2$, offset = 2 bits

alignment

block (or line) The minimum unit of information that can be either present or not present in a cache.

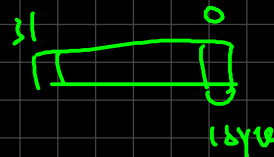
Step 1: byte address → block address (取 cache 的 size)

ex. block = 4 bytes (1 word)

address = 32 bit byte address

address = 0x111112A

block addr. = 0x111112



Step 2: (Block address) modulo (Number of blocks in the cache)

ex. # blocks = 16

index = (0x111112) mod 16

= 12 (也就是 map 到 cache 第 12 (16) = (0001 0010) 2 的 block)

Step 3: 多個 mem address 可能 map 到 相同 的 cache block

∴ 需要額外資訊去 tell 是 不同的 mem addr. ⇒ tag bit

而 tag 就是除了 index & offset 剩下的 higher-order 部分

ex. addr. = 0x111112A
tag index offset



FIGURE 5.17 The three portions of an address in a set-associative or direct-mapped cache. The index is used to select the set, then the tag is used to choose the block by comparison with the blocks in the selected set. The block offset is the address of the desired data within the block.

- D. Consider 32-bit byte address, a direct-mapped cache of size 64K Bytes with each block of size 4 words. A series of memory accesses occur as follows (assume all states are reset initially): 0XDF105670, 0XDF10567C, 0X23A033A0, 0X23A034A0, 0XDF105678.
23. The number of tag bits is (a) 12 bits, (b) 16 bits, (c) 20 bits, (d) 8 bits.
24. The number of blocks in the cache is (a) 4K, (b) 2K, (c) 1K, (d) 8K.
25. The hit access is
 (a) Only 0XDF10567C.
 (b) Only 0X23A034A0.
 (c) 0XDF10567C and 0XDF105678.
 (d) 0XDF10567C, 0X23A034A0 and 0XDF105678.

address = 32 bit $\rightarrow$ #block $\times$ block size
 cache size = 64 kB
 block size = 4 words $\rightarrow$ 1 word = 4 bytes

24. (block offset = 2^4 (16 bytes = 4 word), offset = 4 bit)
 #blocks = $\frac{64 \text{ kB}}{4 \times 4 \text{ B}} = 4 \text{ K}$

25. #block = $4 \times 2^{10} = 2^{12}$, index = 12 bit
 tag = 32 - 12 - 4 = 16 bit
 Index offset

	tag	index	H/M
0XDF105670	DF10	567	M
✓ 0XDF10567C	DF10	567	H ✓
0X23A033A0	23A0	33A	M
0X23A034A0	23A0	34A	M
✓ 0XDF105678	DF10	567	H ✓

4 bytes
 block
 $2 = (2^4)^3 = (16)^3$

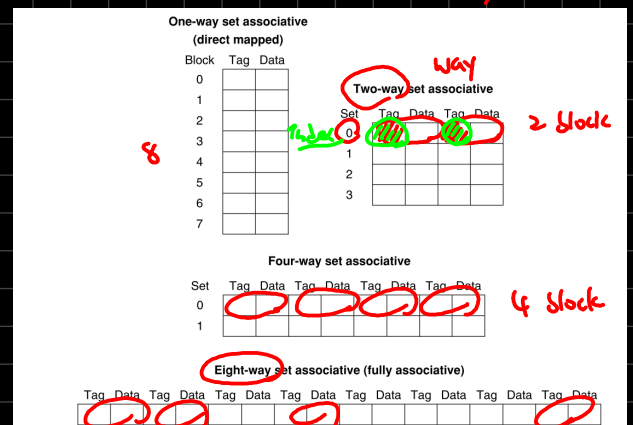
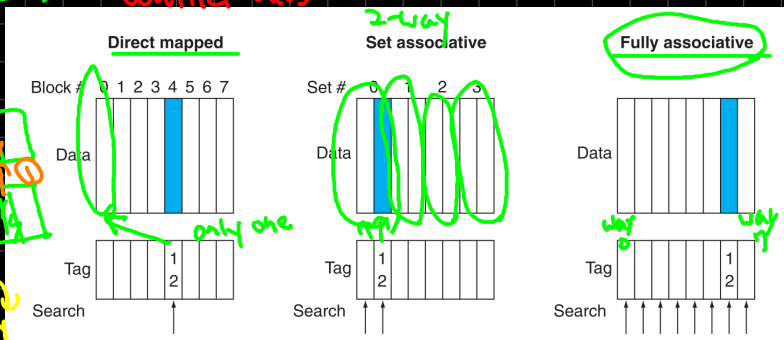
associativity

direct mapped 會出現不同 address map 到同一個 cache block 問題

$\rightarrow$ compete for the same set (也就是 conflict / collision miss)

Solution: 同一個 set 有不同 way 可以填
 conflict miss

find set $\rightarrow$ find way (tag)



(Block number) modulo (Number of sets in the cache)

HW

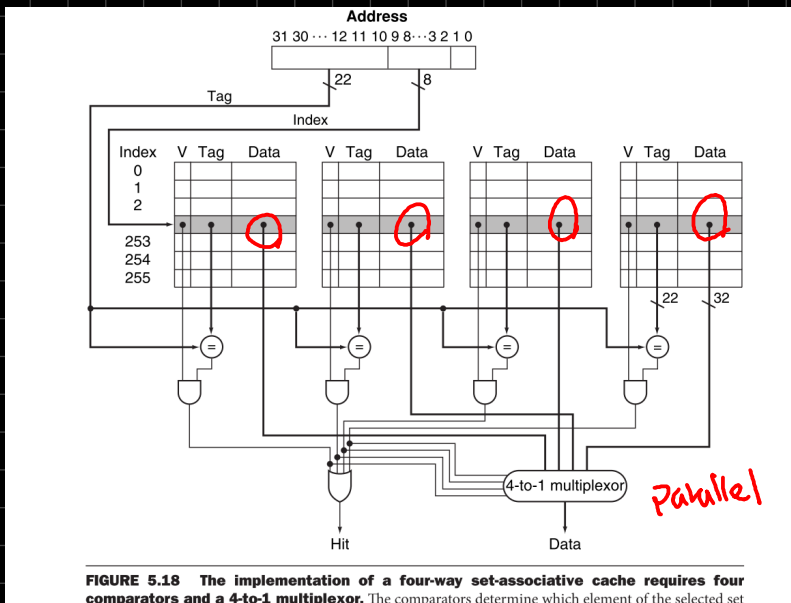


FIGURE 5.18 The implementation of a four-way set-associative cache requires four comparators and a 4-to-1 multiplexor. The comparators determine which element of the selected set

2. (20%) Consider three different cache configurations below:

- Cache 1: direct-mapped with one-word blocks.
- Cache 2: direct-mapped with four-word blocks.
- Cache 3: Two-way set associative with two-word blocks and LRU replacement.

Assuming that each cache has a total data size of 16 32-bit words and all of them are initially empty. 20-bit word address is used.

(a) (3%) What's the size of tag field in one block used in each cache?

(b) (3%) How many total bits are required for each cache, including tag and valid fields?

(c) (12%) Given a series of word-address references: 42, 43, 56, 57, 58, 59, 34, 35, 42, and 43. For caches 2 and 3, please label each reference in the list as a hit or a miss (10%), and show the final cache contents (the word addresses in each block or set) after all references (2%).

(d) (2%) Consider virtual memory system and cache. Please explain alias problem (1%), and the advantage of physically tagged virtually indexed cache (1%).

19
1 byte = 8 bits
cache size = 16 word (1 word = 32 bits)
address = 20 bits word address = 4 x 8 bits = 4 bytes

(a) Cache 1: direct-mapped & 1 word block

$$\# \text{ blocks} = \frac{16}{1} = 16, \text{ index} = 4 \text{ bits}$$

$$\text{block offset} = 2^0, \text{ offset} = 0 \text{ bits}$$

$$\text{tag} = 20 - 4 - 0 = 16 \text{ bits}$$

Cache 2: direct-mapped & 4 word block

$$\# \text{ blocks} = \frac{16}{4} = 4, \text{ index} = 2 \text{ bits}$$

$$\text{block offset} = 2^2, \text{ offset} = 2 \text{ bits}$$

$$\text{tag} = 20 - 2 - 2 = 16 \text{ bits}$$

Cache 3: 2-way assoc. & 2 word block

$$\# \text{ blocks} = \frac{16}{2} = 8, \text{ block offset} = 1 \text{ bit}$$

$$\# \text{ set} = \frac{8}{2} = 4, \text{ index} = 2 \text{ bits}$$

$$\text{block offset} = 2^1, \text{ offset} = 1 \text{ bit}$$

$$\text{tag} = 20 - 2 - 1 = 17 \text{ bits}$$

(b)

The total number of bits in a direct-mapped cache is

$$\# \text{ blocks} \times (\text{block size} + \text{tag size} + \text{valid field size}).$$

direct {

$$\text{cache 1} = 16 \times (1 + 16 + 32) \quad 1 \text{ word} = 4 \text{ byte} = 32 \text{ bits}$$

$$\text{cache 2} = 4 \times (1 + 16 + 4 \times 32) \quad 8 \text{ bytes}$$

$$\text{cache 3} = (2 \times 4) \times (1 + 17 + 2 \times 32)$$

2-way # blocks

